PROVEIT / LEAN PROOF ENGINEERING

Additional Lean Proof Simplifications

Four direct edits and two optional interfaces

Review of the DiophantineConsistency development

26 October 2026

Source snapshot	`29936089835c0c55cca81edc86b4fb0c70ce34d5`
Exclusion baseline	Packets a–u and the incoming directory at the source pin, plus packet v at the final freshness-check pin
Direct result	Five existing theorem proofs shortened in four complete modules; 31 net physical lines removed
Additional result	Exact signed-prefix division and native-rank comparison APIs, supplied separately from the line-saving patches
Deliverables	Independent patches, complete proposed modules, Lean examples, source hashes, audit records, and replay instructions

Interpretation. The earlier proposals are treated as already applied, including optional and unimplemented ideas. The recommendations below are additional to that corpus. Their mathematics uses established library facts; no claim of worldwide mathematical novelty is made. Source-line reductions are measured, while optional APIs carry an explicit additional-code cost.

Contents

1 Results, scope, and the baseline **2**
1.1 What is deliberately excluded . . . 2
1.2 Two kinds of deliverable . . . 3

2 A: Replace signed helical induction by a periodic orbit **3**
2.1 The source obligation . . . 3
2.2 The orbit that matches the hypothesis . . . 3
2.3 Where this helps next . . . 4

3 B: Keep coprime divisibility at the divisibility interface **4**
3.1 The exact theorem . . . 4
3.2 Preserve the degenerate cases . . . 5

4 C: Derive the geometric bound from the exact identity **5**

5 D: Read a positive quadratic margin as an interval **6**
5.1 The existing contradiction proof . . . 6
5.2 A possible later extension . . . 6

6 E: Give the finite signed borrow its exact prefix meaning **7**
6.1 The existing data . . . 7
6.2 Why the signed prefix matters . . . 7
6.3 The old nonnegative theorem as a corollary . . . 8

7 F: Compare native ranks through one scalar coordinate **8**
7.1 The exact affine form . . . 8
7.2 Relation to the present increment theorem . . . 9
7.3 Information that the scalar coordinate does not retain . . . 9

8 Validation and practical adoption **9**
8.1 What the measurements preserve . . . 10
8.2 Recommended sequence . . . 10

9 Further questions **11**

10 Artifact map and reproduction **12**

Primary sources **12**

1 Results, scope, and the baseline

The strongest immediate result is a small collection of proof-only edits with unchanged public statements. Their value is that they remove particular arguments which still remain after the much larger earlier simplification programme. The two accompanying interfaces expose stronger facts about existing definitions; they are candidates for adoption when their additional statements serve multiple clients.

ID	Complete module	Before	After	Removed
A	`WangTiles942HelicalQuotient`	147	136	11
B	`RatioReparameterization`	180	173	7
C	`PolynomialRangeBounds`	150	145	5
D	`MarginSeparation`	141	133	8
	Total	618	587	**31**

Table 1: Physical source-line counts for the four supplied patches. The additional geometric-series import is included. Comments, public declarations, and executable definitions outside the indicated proofs are retained.

These counts do not estimate proof-term size, elaboration speed, or a whole-repository percentage. In particular, a shorter tactic script need not elaborate faster. No timing benchmark is claimed. The package includes both the unmodified target files and their complete replacements, making the count and the scope of each diff directly inspectable.

The review concerns the source tree at the stated commit. Its simplification index lists 21 packets, `2026-10-26a` through `2026-10-26u`. The three archives seen in an earlier directory response were moved into s, t, and u by the final pinned commit. At this pin, `docs/incoming` contains its README; the moved reports were included in the audit. The initial corpus manifest inventories 1,143 files totalling 35,059,944 bytes. This is an inventory count, not a count of independent ideas.

The audit records 120 exclusion entries, with primary report anchors and an index of the 21 manuscripts. Some entries overlap. The ledger includes suggestions in supplementary notes and code prototypes, not just section headings. An idea does not become available again because its implementation is unfinished or its original author recommended deferring it.

Final freshness check. A later upstream check found commit `45ac0dfc366f798d4686070a89d66e8dafdb6879`, 15 commits beyond the tested pin. It adds a twenty-second packet, `2026-10-26v`. That packet was also reviewed, including its manuscript, review notes, and Lean candidates. It concerns finite-state encoding equivalences, machine action and colour proofs, interval propagation, and bond pulses; none duplicates A–F below. The supplemental audit and its source hashes are included separately. Thus the exclusion coverage is **22 packets, a–v**, while the original inventory numbers and 120-entry ledger remain explicitly attributed to the first 21 packets.

All four files patched in this report are unchanged between the two commits. Other production dependencies did change. Accordingly, the compiler evidence and line measurements below remain tied to the original source pin; the later commit is a freshness and exclusion check, not a second claimed build.

1.1 What is deliberately excluded

Earlier packets already cover recursor congruence, scanner invariants, successful decoder graphs, assignment naturality, witness-layout transport, finite residual sums, generated power certifi-

cates, common circuit tails, coefficient majorants, exact local natural carry decompositions, cyclic carry fixed points, Pell sign relations and approximation bounds, internal-model induction, compositional formula specifications, and shared-edge Wang rows. They also explicitly include max-plus representation-depth certificates, semantic renaming chains, and cyclic packing via a monoid-algebra homomorphism. None is counted among the results here.

The new arithmetic edits are intentionally described by their exact targets. "Use the library" is background practice, not a new research contribution. For example, proposal B concerns coprime cancellation in the particular sum $QF + JT$; it does not claim fresh credit for the already suggested general preference to avoid unnecessary quotient witnesses.

1.2 Two kinds of deliverable

- **A–D are direct replacement patches.** They change five proof bodies in four existing modules and add one explicit import. Their existing theorem statements and data definitions are preserved.
- **E–F are optional interfaces.** They strengthen the mathematical description of the current signed borrow and rank definitions. They are supplied as additional Lean modules, not included in the 31-line saving. Their total code cost is positive until enough clients benefit.

2 A: Replace signed helical induction by a periodic orbit

2.1 The source obligation

In `WangTiles942HelicalQuotient`, `row_zero_of_helix` assumes a helical period

$$T(x + w, t - 1) = T(x, t)$$

for an arbitrary $T : \mathbb{Z} \to \mathbb{Z} \to \alpha$, and proves

$$T(x, t) = T(x + wt, 0).$$

The current proof performs integer induction, with separate positive-time and negative-time branches and a coordinate rearrangement in each branch. Those branches establish a fact already captured by the library's closure of periods under integer multiplication.

2.2 The orbit that matches the hypothesis

Fix x, t, and define

$$f(k) = T(x + wk, t - k).$$

The helical premise gives $f(k + 1) = f(k)$, so f has period 1. Therefore it has period t, even when $t < 0$. Evaluating that period at zero gives $f(t) = f(0)$, exactly the required equality after simplification.

The replacement proof is:

```
have hp : Function.Periodic
    (fun k : ℤ => T (x+w*k) (t-k)) 1 := by
  intro k
  simpa only [mul_add, mul_one, add_assoc,
    sub_add_eq_sub_sub] using hhelix (x+w*k) (t-k)
simpa using (hp.int_mul t).eq.symm
```

There is no new domain restriction. The width w can be zero or negative, the time is any integer, and α needs no structure. The proof does not require a nonzero period, a finite codomain, a unique Start tile, a computation trace, or coprimality of geometric parameters.

The full declaration decreases from 19 to 8 lines, and the module from 147 to 136. The public header is identical. This is distinct from the earlier proposal to use integer induction centred on a known interval point: here an entire signed induction is eliminated by identifying a one-dimensional periodic orbit.

2.3 Where this helps next

The existing downstream factorization through a finite marked word can use the unchanged theorem. Future helical coordinate identities should first ask whether the change of coordinates defines an orbit of an already known period. This is a useful local selection rule, not a recommendation to introduce a new group-action framework for the current eight-line proof.

3 B: Keep coprime divisibility at the divisibility interface

3.1 The exact theorem

In `RatioReparameterization`, the premise $J \mid F-1$ implies that F and J are coprime. The target is

$$JF \mid QF + J(H-C) \quad \Longleftrightarrow \quad J \mid Q \ \wedge \ F \mid H-C.$$

The existing proof explicitly builds divisibility witnesses, subtracts a known divisible summand twice, and constructs a combined quotient in the reverse direction. The mathematical argument is already available through a short chain of divisibility lemmas.

Set $T = H - C$. If $JF \mid QF + JT$, then both J and F divide the sum. Since $J \mid JT$, subtraction gives $J \mid QF$, and coprimality cancels F. Similarly, $F \mid QF$ leaves $F \mid JT$, and coprimality cancels J. Conversely, $J \mid Q$ gives $JF \mid QF$, and $F \mid T$ gives $JF \mid JT$; add them.

```
have hcop := isCoprime_of_congruence hJF
constructor
· intro h
  exact ⟨hcop.symm.dvd_of_dvd_mul_right
    ((dvd_add_left (dvd_mul_right J _)).mp
      ((dvd_mul_right J F).trans h)),
    hcop.dvd_of_dvd_mul_left
    ((dvd_add_right (dvd_mul_left F Q)).mp
      ((dvd_mul_left F J).trans h))⟩
· rintro ⟨hQ, hT⟩
  exact dvd_add (mul_dvd_mul_right hQ F)
    (mul_dvd_mul_left J hT)
```

The display is line-wrapped for print; the supplied source is the measured 11-line declaration. The old declaration has 18 lines. No helper or import is added, and the complete module removes seven lines.

3.2 Preserve the degenerate cases

No positivity or nonzero premise should be added to make the proof easier. The existing assumption permits cases such as $J = 0, F = 1$, and $F = 0, J = 1$ or -1. The abstract divisibility proof covers these cases without a separate branch. Replacing it by an argument that divides by J or F would need extra bookkeeping and could accidentally weaken the theorem.

This proposal does not change `newIndex`, `oldIndex`, the ratio-register expressions, or the source equation. It removes explicit witnesses only inside a proposition-valued theorem. The original and replacement proofs use the same standard axiom set, `propext` and `Quot.sound`.

4 C: Derive the geometric bound from the exact identity

The first two theorems in `PolynomialRangeBounds` currently prove an exact geometric-sum identity and a coarse estimate by two separate inductions. The first theorem is precisely the library theorem `mul_geom_sum`; the second follows directly from that identity and nonnegativity.

Write $S_d = \sum_{i<d} B^i$. The identity is

$$(B-1)S_d = B^d - 1.$$

It holds for every integer B, including zero, one, and negative values. For the estimate, assume $B \geq 2$. Then $S_d \geq 0$ and $B - 1 \geq 1$, so

$$S_d \leq (B-1)S_d = B^d - 1.$$

Consequently,

$$S_{d+1} = S_d + B^d \leq 2B^d - 1 < 2B^d.$$

The same reasoning includes $d = 0$; there is no exceptional base-case argument to maintain.

```
theorem geometric_identity (B : ℤ) (d : ℕ) :
    (B - 1) * (∑ i ∈ range d, B ^ i) = B ^ d - 1 :=
  mul_geom_sum B d

-- Replacement body of geometric_lt_two_top:
  have hg := geometric_identity B d
  have hs : 0 ≤ ∑ i ∈ range d, B ^ i :=
    sum_nonneg (fun _ _ => by positivity)
  have h := mul_le_mul_of_nonneg_right
    (show 1 ≤ B - 1 by omega) hs
  rw [sum_range_succ]
  nlinarith
```

The two declarations decrease from 7 and 9 lines to 3 and 7. The patch adds an explicit import of `Mathlib.Algebra.Ring.GeomSum`, making the net module saving five lines rather than six. The explicit import documents the new dependency and avoids relying on an incidental transitive import.

The coefficient bounds, polynomial support, and evaluation estimates remain unchanged. This matters for the user's exclusion rule: earlier majorant and support-envelope proposals are not being resubmitted. The additional observation is that the elementary geometric estimate can share the exact identity already immediately above it.

5 D: Read a positive quadratic margin as an interval

5.1 The existing contradiction proof

In `MarginSeparation`, `coordinate_gt_of_margin_pos` assumes

$$K > 0, \quad Y \geq 2, \quad \text{cap} > 0, \quad g \geq 0,$$

and

$$0 < (K^2 - 4(C - KY)^2)\text{cap} - 8gK^2.$$

It concludes $K < C$. The old proof first obtains positivity of the quadratic gap, then assumes $C \leq K$, manufactures a lower bound for $-(C-KY)$, and uses square monotonicity to contradict the gap.

The direct interpretation of the positive quadratic is an interval centred on KY. Discarding the nonnegative tail and cancelling the positive cap gives

$$4(C - KY)^2 < K^2.$$

Since $K > 0$, the library's square-to-interval theorem gives

$$-K < 2(C - KY) < K.$$

Only the lower endpoint is needed. Combining it with $KY \geq 2K$ yields

$$2C > 2KY - K \geq 3K > 2K,$$

so $C > K$.

```
have hgap : 0 < K ^ 2 - 4 * (C - K * Y) ^ 2 := by
  apply (mul_pos_iff_of_pos_right hcap).mp
  exact lt_of_lt_of_le hm (sub_le_self _ (by positivity))
have hwindow := abs_lt_of_sq_lt_sq'
  (a := 2 * (C - K * Y)) (b := K)
  (by nlinarith only [hgap]) hK.le
have hKY := mul_le_mul_of_nonneg_left hY hK.le
nlinarith only [hwindow.1, hKY, hK]
```

The declaration decreases from 19 to 11 lines, and the module from 141 to 133. The complete public statement is retained. The positive cap and nonnegative tail remain explicit; there is no appeal to a stronger source theorem or an assumed positivity result for a constructed model.

5.2 A possible later extension

If several clients need both endpoints, a small theorem exposing the interval would be useful. That is not needed for this patch, and introducing such a helper now would reduce its net saving. The delivered change therefore uses the existing library fact directly. This is also separate from earlier rational-root descent and minimum-saturation proposals: it changes the proof of a concrete geometric implication within the margin argument.

6 E: Give the finite signed borrow its exact prefix meaning

6.1 The existing data

`SparseBorrowCarry` defines a borrow sequence by

$$b_0 = 0, \qquad b_{i+1} = \begin{cases} -1, & a_i + b_i < 0, \\ 0, & a_i + b_i \geq 0, \end{cases}$$

and normalized digits

$$d_i = a_i + b_i - B b_{i+1}.$$

Under $B > 1$ and $-B < a_i < B$ for $i < n$, the existing proofs establish $0 \leq d_i < B$, the natural word $W_n = \sum_{i<n} d_i B^i$, its bound $W_n < B^n$, and the telescoping equality

$$W_n + B^n b_n = S_n, \qquad S_n = \sum_{i<n} a_i B^i.$$

These are exactly the uniqueness hypotheses for integer Euclidean division.

Proposition 6.1 (Signed-prefix division)**.** *With the preceding hypotheses,*

$$S_n \operatorname{div} B^n = b_n, \qquad S_n \bmod B^n = W_n.$$

These statements hold even when $S_n < 0$.

Proof. Since $B^n > 0$, the equality $W_n + B^n b_n = S_n$ and bounds $0 \leq W_n < B^n$ uniquely determine the integer quotient and nonnegative remainder. Apply `Int.ediv_emod_unique` to the existing word-cast, digit-sum, and word-bound lemmas. □

The supplied `SignedPrefix.lean` imports the actual production module. Its main theorem is proved by:

```
apply (Int.ediv_emod_unique (by positivity)).mpr
refine ⟨?_, by positivity, ?_⟩
· rw [word_cast (by omega) a ha]
  exact digit_sum B n a
· exact_mod_cast word_lt hB a ha
```

6.2 Why the signed prefix matters

Earlier packets already propose extracting quotient and remainder from a normalized natural local carry equation. They also cover cyclic carries with a different global boundary condition. The new contract here is specifically the *whole finite signed prefix*, with denominator B^n and terminal borrow b_n. It uses the fixed initial borrow and finite telescoping identity, and remains meaningful for negative total values. No generic claim that quotient uniqueness is new is intended.

For example, with $B = 4, n = 2, a_0 = -1, a_1 = 0$, the total is $S_2 = -1$, the word is $W_2 = 15$, and $b_2 = -1$. Thus

$$-1 = 15 + 16(-1), \qquad -1 \bmod 16 = 15.$$

Truncating the signed total to `toNat` would lose this information: (-1).toNat $= 0$, not 15. The package contains a kernel-checked example of this boundary behaviour. It also checks $n = 0$, where the denominator is one, and both word and borrow are zero.

6.3 The old nonnegative theorem as a corollary

The current recurrence gives $b_n \in \{-1, 0\}$. If $S_n \geq 0$, integer division by $B^n > 0$ is nonnegative, so the quotient characterization forces $b_n = 0$. Conversely, $b_n = 0$ makes $S_n = W_n \geq 0$. Hence

$$b_n = 0 \quad \iff \quad S_n \geq 0.$$

The existing nonnegative word theorem is then a short corollary: substitute $b_n = 0$ in the telescoping identity and apply `Int.toNat`.

Three theorems are supplied: `prefix_divmod`, `borrow_zero_iff`, and `word_eq_toNat_of_nonneg`. They do not change the recurrence or decoder. The new file adds code, and is not part of the reported saving. Adoption is attractive if signed prefix comparisons, concatenation, or modular word clients need the exact quotient. For the single existing `word_eq_toNat` client alone, retaining its current compact proof is reasonable.

7 F: Compare native ranks through one scalar coordinate

7.1 The exact affine form

The newer `NativeRankVariation` defines

$$R(C, F) = (q(q - F) - C)(q^2 - 1) + (M_C + qM_F)J.$$

For fixed q, J, M_C, M_F, set

$$U(C, F) = C + qF, \quad s = q^2 - 1, \quad K_0 = (M_C + qM_F)J.$$

Then

$$R(C, F) = (q^2 - U(C, F))s + K_0.$$

When $q > 1$, we have $s > 0$, and rank comparison is exactly reversed comparison of U.

Proposition 7.1 (Exact comparison). *For fixed $q, J, M_C, M_F \in \mathbb{Z}$ with $q > 1$,*

$$\begin{aligned} R(C', F') < R(C, F) &\iff U(C, F) < U(C', F'), \\ R(C', F') \leq R(C, F) &\iff U(C, F) \leq U(C', F'), \\ R(C', F') = R(C, F) &\iff U(C', F') = U(C, F). \end{aligned}$$

Proof. Rewrite both ranks in affine form. Cancel the common additive term, cancel multiplication by the positive factor $q^2 - 1$, and use the order reversal of subtraction from the same constant. Equality only needs that factor to be nonzero. □

The strict-comparison proof in the actual-source adapter is:

```
have hs : 0 < q^2-1 := by nlinarith
rw [windowRank_affine, windowRank_affine,
  add_lt_add_iff_right, mul_lt_mul_iff_of_pos_right hs,
  sub_lt_sub_iff_left]
```

7.2 Relation to the present increment theorem

The current theorem considers correlated updates

$$C' = C + \delta, \qquad F' = F + A\delta.$$

The scalar coordinate changes by

$$U(C', F') - U(C, F) = \delta(1 + qA).$$

Thus the existing hypotheses $\delta > 0$, $A \geq 0$, $q > 1$ are one sufficient way to increase U, and hence decrease the rank. The exact comparison API admits other updates as long as their aggregate coordinate increases. It separates the order argument from the particular parameterization of a proposed mutation.

The supplied `RankPotentialActual.lean` imports the production `windowRank`. It also gives `core_rank_lt_iff` and `core_rank_eq_iff` for the actual `Carrier65.core` fronts. These adapters require equality of the two q values and equality of J; using the same port vector fixes the two mask ports. They use the existing `core_rank` bridge, preserving its source high-mask convention.

7.3 Information that the scalar coordinate does not retain

The transformation is not injective on coefficient pairs. For any integer k,

$$U(C + qk, F - k) = U(C, F).$$

Accordingly, distinct pairs can have equal rank. The API is not an order isomorphism from pairs to integers, and even as a scalar affine map its integer image need not be all of $\mathbb{Z}$. At $q = 1$, the coefficient $q^2 - 1$ vanishes and all pairs have the same rank; the supplied file explicitly retains this degeneration.

The comparison theorem requires ordinary integer equalities. A congruence for F modulo q does not determine U and is insufficient. More substantially, this algebra does not construct a valid change of the encoded word, prove digit bounds after an update, establish a carry invariant, or supply a global controller. Those remain separate obligations. The existing rank-variation module correctly states this distinction, and the proposed API preserves it.

The previous packet u proves a stronger *lower magnitude bound* for the native rank from positive source assumptions. The present proposal instead compares two ranks through an exact affine coordinate. It is additional to that bound. The current variation proofs are already short, so this API has no measured immediate line saving. It becomes useful when multiple controller or equality clients would otherwise rederive the same order cancellation.

8 Validation and practical adoption

The checked source pin is `29936089835c0c55cca81edc86b4fb0c70ce34d5`, using Lean 4.32.0 (compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`) and mathlib `81a5d257c8e4 10db227a6665ed08f64fea08e997`. The relevant project dependency closure was rebuilt from source: 62 project modules and one otherwise missing mathlib module. The remaining mathlib and package objects were reused from a cache matching the pinned dependency manifest. All 63 recorded compilations completed successfully and produced nonempty objects.

The archive's own replay script then completed the following checks:

- All four complete proposed production modules compiled successfully in a fresh object overlay of that baseline.
- The five changed theorem headers matched their original headers exactly, and baseline and replacement `\#print axioms` reports were identical.
- The existing `WangTiles942HelicalQuotientChecks` and `RatioMarginFinalizer` clients compiled against the proposed modules.
- The two optional modules, `SignedPrefix` and `RankPotentialActual`, compiled against the actual production definitions. The signed-prefix file checks a negative total and an empty prefix; the rank file includes the equal-rank family and the degenerate base-one case.
- The separate helical and arithmetic proof examples also compiled. No checked axiom report contains `sorryAx`. The extra helical endpoint audit retained its original axiom lists as well.

The periodic-orbit and coprime-divisibility results depend on `propext` and `Quot.sound`. The geometric and margin results also use `Classical.choice`; these are exactly their original axiom lists. The full command receipts, source hashes, output logs, and line counts are included. Recorded elapsed times describe these runs only and are not before/after performance measurements. Validation is against the stated source pin, not against the newer freshness-check commit described in Section 1.

8.1 What the measurements preserve

The four direct patches retain the exact public theorem headers before `:=`. Their diffs leave every executable definition intact. The geometric import is the only import change. For the five changed proofs, the package records baseline and replacement axiom reports and checks their equality. No custom axiom or admitted proof is introduced.

The checks establish that the indicated full modules and selected clients elaborate and are accepted by the pinned Lean kernel. They do not establish a clean rebuild of every DiophantineConsistency module, a formal proof of the metatheory of ZFC, or any of the project's open equation-completeness obligations. The alterations concern proofs of already stated intermediate results.

Axiom lists alone are not semantic equivalence checks: a theorem with a changed hypothesis could have the same axiom list. That is why the bundle also verifies unchanged theorem headers and provides complete source diffs. Conversely, line counts alone do not show that code was checked; the compiler logs and source hashes identify what was compiled.

8.2 Recommended sequence

1. Apply A–D independently, retaining all public names and the new explicit geometric-series import. These are small changes with exact replacement files and minimal dependencies.
2. Rebuild the indicated modules and the focused downstream clients using the pinned toolchain. Keep the original endpoint statements as the integration contract.
3. Add E only if at least one additional signed-prefix or modular-word client needs its stronger specification. Otherwise keep it as a checked research module.
4. Add F when comparisons of different native fronts become frequent. Use the exact scalar comparison theorem, and keep word-validity and mutation-existence hypotheses at their own interface.

Nothing in this report requires editing the generated arithmetic circuits, the decimal equation, its variable count, or any theorem statement defining the final consistency target. The proposed changes are delivered for review; they were not pushed to the repository.

9 Further questions

The following questions extend the new interfaces rather than reintroducing earlier proposals.

1. **Signed-prefix concatenation.** Can the exact quotient/remainder contract yield a useful concatenation theorem for the existing borrow word, including the effect of a nonzero incoming borrow? The key condition is the signed prefix boundary, not a generic local carry equation.
2. **Canonical comparison clients.** Which native-controller steps can prove an increase in $C+qF$ directly while preserving the source ports? Before adopting the new API widely, measure the combined cost of the order theorem and at least two real clients.
3. **Further helical orbit proofs.** Are there other signed-coordinate proofs in the newly built Wang construction that become a library periodicity statement after choosing a one-parameter orbit? A useful candidate should eliminate a complete argument, not merely rename an induction.
4. **Margin endpoints.** Do later finalizer arguments use the upper margin endpoint $2(C-KY) < K$ as well as the lower one? If so, exposing both endpoints may justify a shared lemma; the current patch intentionally avoids paying that cost for a single consumer.
5. **Simp boundaries.** Should the exact rank-comparison lemmas be used explicitly or locally as simp rules? They expose a useful normal form, but a global orientation could unfold intended rank abstractions in unrelated goals. A local client experiment should decide this.

10 Artifact map and reproduction

Path	Purpose
`report/report.tex`	Complete manuscript source, with its generated validation summary.
`report/report.pdf`	Rendered and visually inspected manuscript.
`patches/*.patch`	Four independent production-source patches, plus a combined diff.
`baseline/`	The four exact target modules at the pinned commit.
`proposed/`	Complete patched versions of those four modules.
`lean/SignedPrefix.lean`	Actual-source signed-prefix quotient, remainder, sign classification, and boundary examples.
`lean/RankPotentialActual.lean`	Actual-source exact rank comparison and Carrier65 adapters.
`lean/AlgebraDeltas.lean`	Small independent arithmetic proof examples.
`lean/HelicalOrbit.lean`	Small independent helical proof example.
`evidence/`	Initial 21-packet audit plus the packet-v supplement, exclusion ledger, corpus manifest, line counts, hashes, and validation receipts.
`logs/`	Compiler and axiom-audit output for the shipped proofs.
`scripts/`	Header/count verification and focused replay tools.

The README gives the exact commands and prerequisites. Reproduction uses Lean 4.32.0 and the repository-pinned mathlib revision. The archive does not contain a toolchain, mathlib binaries, full historical source snapshots, or the repository's compiled objects. It includes the complete changed files and enough metadata to retrieve the exact external sources.

Primary sources

1. ProveIt, DiophantineConsistency, pinned source tree. All compiled production-source claims in this report refer to commit `29936089835c0c55cca81edc86b4fb0c70ce34d5`.
2. The 21 placed proof-simplification packets and their adoption record. The supplied exclusion ledger gives finer anchors.
3. Incoming README at the same pin, including the packet-placement table.
4. Packet v at the final freshness-check commit, together with the source comparison. Its audit supplements the original 21-packet record.
5. Mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`: periodicity, geometric sums, coprimality, and ordered-ring square and absolute-value facts.
6. Lean 4.32.0, compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`: integer Euclidean division, in particular `Int.ediv_emod_unique`.